

Métodos de Sequelize

1) create

create es un método que se utiliza para **insertar un nuevo registro** en la base de datos. Es una forma rápida y sencilla de crear un registro en una tabla usando un objeto que contiene las propiedades (o columnas) y sus valores. Este método crea el registro y lo guarda en la base de datos en una sola operación.

Sintaxis:

```
Model.create({
  columna1: valor1,
  columna2: valor2,
  // otras columnas y valores...
})
```

Ejemplo simple:

Supongamos que tienes un modelo User y quieres crear un nuevo usuario con un nombre y un email:

```
User.create({
  name: 'John Doe',
  email: 'john.doe@example.com'
})
```

En este ejemplo:

- name y email son las columnas en la tabla User.
- 'John Doe' y 'john.doe@example.com' son los valores que estamos asignando a esas columnas.

2) update

update es un método utilizado para **actualizar registros** en la base de datos que cumplen con ciertas condiciones. Permite modificar una o varias columnas de uno o varios registros en una tabla. A diferencia de otros métodos como create o destroy, update no devuelve los registros actualizados directamente, sino que devuelve información sobre cuántos registros fueron afectados.

Sintaxis:

```
Model.update({
  columna1: nuevoValor1,
  columna2: nuevoValor2,
  // otras columnas y valores...
}, {
  where: {
    // Condiciones para filtrar los registros que se deben
    actualizar
  }
})
```

- El primer objeto define las columnas y los nuevos valores que se quieren asignar.
- El segundo objeto especifica la condición (where) que determina qué registros se van a actualizar.

Ejemplo simple:

Supongamos que tienes un modelo User y quieres actualizar el email de un usuario cuyo id es 5:

```
User.update(
  { email: 'nuevo.email@example.com' },
  { where: { id: 5 } }
)
```

En este ejemplo:

- **email:** Es la columna que estamos actualizando.
- **'nuevo.email@example.com':** Es el nuevo valor que asignamos a la columna email.
- **where: { id: 5 }:** Es la condición que filtra el registro que queremos actualizar (en este caso, el usuario con id igual a 5).

Nota importante:

update devuelve un array en el que el primer elemento indica cuántos registros fueron afectados por la actualización. No devuelve los registros actualizados en sí mismos. Si deseas obtener los registros actualizados, debes hacer una consulta adicional usando findOne o findAll después de la actualización.

3) destroy

destroy es un método utilizado para **eliminar registros** de la base de datos que cumplen con ciertas condiciones. Es equivalente a un **DELETE** en SQL y permite borrar uno o varios registros de una tabla en función de las condiciones que especifiques.

Sintaxis:

```
Model.destroy({
  where: {
    // Condiciones para filtrar los registros que se deben eliminar
  }
})
```

- El objeto where especifica las condiciones que determinan qué registros serán eliminados.

Ejemplo simple:

Supongamos que tienes un modelo User y quieres eliminar un usuario cuyo id es 5:

```
User.destroy({
  where: {
    id: 5
  }
})
```

En este ejemplo:

- **where: { id: 5 }:** Es la condición que filtra el registro que queremos eliminar (en este caso, el usuario con id igual a 5).

Notas importantes:

- **Retorno:** destroy devuelve un número que indica cuántos registros fueron eliminados. Si no se encuentra ningún registro que cumpla con las condiciones, devuelve 0.
- **Condiciones:** Es importante utilizar el objeto where para evitar eliminar todos los registros de la tabla. Si no se proporciona una condición where, Sequelize eliminará todos los registros de la tabla, lo cual es peligroso y generalmente no se recomienda.

4) findAll

findAll es un método utilizado para realizar consultas en la base de datos y recuperar múltiples registros de una tabla. Es equivalente a un SELECT en SQL, y te permite obtener todos los registros que cumplen con ciertas condiciones. Este método devuelve un array con los resultados encontrados.

La sintaxis básica es:

```
Model.findAll({
  where: {
    // Condiciones para filtrar los resultados
  },
  attributes: ['columna1', 'columna2'], // Seleccionar sólo ciertas
columnas
  order: [['columna', 'ASC']], // Ordenar los resultados
  limit: 10, // Limitar la cantidad de resultados
  offset: 0, // Desplazamiento para paginación
})
```

Ejemplo simple:

```
User.findAll({
  where: {
    status: 'activo'
  }
})
```

Parámetros comunes

- **where:** Especifica las condiciones que deben cumplir los registros (por ejemplo, { age: { [Op.gt]: 18 } } para mayores de 18 años).
- **attributes:** Define las columnas que se quieren obtener. Si se omite, se obtienen todas.
- **order:** Ordena los resultados en función de una columna (**ASC** para ascendente, **DESC** para descendente).
- **limit:** Limita la cantidad de registros que se recuperan.
- **offset:** Define desde qué registro se empieza a contar, útil para paginación.

5) findOne

findOne es un método utilizado para realizar consultas en la base de datos y recuperar un solo registro que cumpla con ciertas condiciones especificadas. Es similar a findAll, pero en

lugar de devolver un array con múltiples registros, devuelve solo el primer registro que coincide con las condiciones, o null si no se encuentra ningún registro.

Sintaxis básica:

```
Model.findOne({
  where: {
    // Condiciones para filtrar el resultado
  },
  attributes: ['columna1', 'columna2'], // Seleccionar solo ciertas
columnas
  order: [['columna', 'ASC']], // Ordenar el resultado
})
```

Ejemplo simple:

Supongamos que tienes un modelo User y quieres obtener el primer usuario cuyo email sea user@example.com:

```
User.findOne({
  where: {
    email: 'user@example.com'
  }
})
```

Parámetros comunes:

- **where:** Especifica las condiciones que debe cumplir el registro (por ejemplo, { username: 'john_doe' }).
- **attributes:** Define las columnas que se quieren obtener. Si se omite, se obtienen todas.
- **order:** Puedes ordenar los resultados en función de una columna (ASC para ascendente, DESC para descendente). Esto es útil si, por ejemplo, quieres obtener el registro más reciente.

6) findByPk

findByPk es un método utilizado para **encontrar un registro en la base de datos basado en su clave primaria (primary key)**. Es un método conveniente y directo para obtener un registro específico cuando ya conoces el valor de su clave primaria. Si no se encuentra ningún registro con esa clave primaria, el método devuelve null.

Sintaxis:

```
Model.findByPk(id, {
```

```
attributes: ['columna1', 'columna2'], // Seleccionar solo ciertas
columnas (opcional)
}))
```

- **id:** Es el valor de la clave primaria del registro que deseas buscar.
- **attributes (opcional):** Define las columnas que se quieren obtener. Si se omite, se obtienen todas las columnas.

Ejemplo simple:

Supongamos que tienes un modelo User y quieres encontrar un usuario específico cuyo id sea 5:

```
User.findByPk(5)
```

Como se basa en la clave primaria, que suele ser un campo indexado en la base de datos, las consultas usando findByPk tienden a ser rápidas.

findByPk no requiere un objeto where, ya que el valor de la clave primaria es suficiente para identificar el registro.

7) findOrCreate

findOrCreate es un método que se utiliza para **buscar un registro** en la base de datos y, **si no existe, lo crea**. Es útil cuando quieres asegurarte de que un registro exista sin necesidad de realizar una búsqueda y una inserción por separado, optimizando así la operación. Este método devuelve un array con dos elementos: la instancia del registro encontrado o creado y un booleano que indica si el registro fue creado (true) o encontrado (false).

Sintaxis:

```
Model.findOrCreate({
  where: {
    // Condiciones para buscar el registro
  },
  defaults: {
    // Valores a usar si se crea un nuevo registro
  }
}))
```

- **where:** Especifica las condiciones para buscar un registro existente.
- **defaults:** Define los valores que se usarán si no se encuentra un registro y se necesita crear uno nuevo.

Ejemplo simple:

Supongamos que tienes un modelo User y quieres asegurarte de que exista un usuario con un email específico:

```
User.findOrCreate({
  where: { email: 'user@example.com' },
  defaults: {
    name: 'John Doe',
    age: 30
  }
})
```

En este ejemplo:

- **where: { email: 'user@example.com' }:** Es la condición para buscar un registro con ese email.
- **defaults:** Si no se encuentra un usuario con ese email, se crea uno nuevo con name como 'John Doe' y age como 30.

El método findOrCreate combina dos operaciones en una sola, haciendo la búsqueda y la creación de manera eficiente.

8) count

count es un método utilizado para **contar el número de registros** que cumplen con ciertas condiciones en una tabla de la base de datos. Es similar a un **SELECT COUNT(*)** en SQL, y es útil cuando solo necesitas saber cuántos registros coinciden con una condición específica sin necesidad de traer todos los datos.

Sintaxis:

```
Model.count({
  where: {
    // Condiciones para filtrar los registros que se deben contar
  }
})
```

- **where:** Especifica las condiciones que deben cumplir los registros para ser contados.

Ejemplo simple:

Supongamos que tienes un modelo User y quieres contar cuántos usuarios tienen el estado activo:

```
User.count({
  where: {
    status: 'activo'
  }
})
```

En este ejemplo:

- `where: { status: 'activo' }`: Es la condición que se utiliza para contar solo los registros que tengan el estado activo.
- El método devuelve el número de registros que cumplen con esa condición.

Parámetros adicionales:

- **attributes**: Puedes contar en base a una columna específica o contar valores distintos (DISTINCT).
- **group**: Puedes agrupar los resultados y contar el número de registros en cada grupo. Esto es útil si quieres saber, por ejemplo, cuántos usuarios hay en cada país o ciudad.
- **include**: Si trabajas con asociaciones entre modelos, puedes incluir modelos relacionados en el conteo.

Ejemplo con group:

Si quieres contar cuántos usuarios hay por cada estado (activo o inactivo):

```
User.count({
  attributes: ['status'],
  group: 'status'
})
```

En este caso, `counts` será un objeto con los diferentes estados como clave y el número de registros que pertenecen a cada uno como valor.

Ejemplo con asociaciones:

Si tienes un modelo `Post` que está relacionado con un modelo `Comment`, y quieres contar cuántos comentarios hay para un post específico:

```
Comment.count({
  where: {
    postId: 1 // Contar comentarios del post con ID 1
  }
})
```


Opciones que se utilizan en los métodos de consulta de Sequelize

9) include

include es una opción que se utiliza en los métodos de consulta como findAll, findOne, findByPk, count, entre otros, **para incluir asociaciones o relaciones con otros modelos en la consulta**. Permite obtener datos relacionados de otros modelos (tablas) que tienen una relación establecida, como belongsTo, hasMany, hasOne, o belongsToMany. Esto es útil para hacer consultas complejas que traen datos de múltiples tablas en una sola operación.

Ejemplo de uso básico

Supongamos que tienes dos modelos: User y Post, donde un User tiene muchos Post (User.hasMany(Post)). Si quieres obtener un usuario junto con todos sus posts, puedes hacerlo así:

```
User.findOne({
  where: { id: 1 },
  include: [{
    model: Post
  }]
})
```

En este ejemplo:

- model: Post: Especifica el modelo relacionado que quieres incluir en la consulta.
- Sequelize hará un JOIN entre las tablas User y Post para obtener el usuario con ID 1 y sus posts asociados.

Ejemplo con belongsToMany:

Si tienes una relación muchos a muchos, por ejemplo, entre User y Group (un usuario puede pertenecer a muchos grupos y un grupo puede tener muchos usuarios), puedes hacer lo siguiente:

```
User.findAll({
  include: [{
    model: Group,
    through: { attributes: [] } // Omite las columnas de la tabla
intermedia
```

```
    ]]  
  })
```

En este caso, Sequelize hará un JOIN para traer todos los grupos a los que pertenece cada usuario.

10) where

where es una opción que se utiliza en métodos de consulta como findAll, findOne, findByPk, update, destroy, y otros, para especificar las condiciones o filtros que deben cumplir los registros que se buscan, actualizan o eliminan en la base de datos. Es equivalente a la cláusula **WHERE** en SQL y permite realizar consultas muy precisas y específicas.

Ejemplo básico:

Supongamos que tienes un modelo User y quieres buscar todos los usuarios cuyo estado sea activo:

```
User.findAll({  
  where: {  
    status: 'activo'  
  }  
})
```

En este ejemplo:

- **where: { status: 'activo' }**: Especifica la condición para buscar solo los registros donde la columna status tenga el valor activo.

Operadores en where:

Sequelize ofrece una serie de operadores (disponibles a través del objeto Op) para hacer condiciones más complejas en las consultas. Algunos ejemplos incluyen:

- **Op.eq**: Igual a (=)
- **Op.ne**: Diferente de (!=)
- **Op.gt**: Mayor que (>)
- **Op.gte**: Mayor o igual que (>=)
- **Op.lt**: Menor que (<)
- **Op.lte**: Menor o igual que (<=)
- **Op.in**: En un conjunto de valores (IN)
- **Op.notIn**: No en un conjunto de valores (NOT IN)
- **Op.like**: Coincidencia con un patrón (LIKE)

Ejemplos con operadores:

1. Usuarios mayores de 30 años

```
User.findAll({
  where: {
    age: {
      [Op.gt]: 30
    }
  }
})
```

[Op.gt]: 30: Busca usuarios cuya edad sea mayor a 30.

2. Usuarios con email que contenga "example.com"

```
User.findAll({
  where: {
    email: {
      [Op.like]: '%@example.com'
    }
  }
})
```

[Op.like]: '%@example.com': Busca usuarios cuyo email termine en @example.com.

3. Usuarios con estado activo o inactivo

```
User.findAll({
  where: {
    status: {
      [Op.in]: ['activo', 'inactivo']
    }
  }
})
```

[Op.in]: ['activo', 'inactivo']: Busca usuarios cuyo estado sea activo o inactivo.

Combinación de múltiples condiciones

Puedes combinar múltiples condiciones utilizando operadores lógicos como Op.and y Op.or:

Ejemplo con Op.and

Buscar usuarios activos mayores de 30 años:

```
User.findAll({
  where: {
    [Op.and]: [
      { status: 'activo' },
      { age: { [Op.gt]: 30 } }
    ]
  }
})
```

Ejemplo con Op.or

Buscar usuarios que sean administradores o mayores de 50 años:

```
User.findAll({
  where: {
    [Op.or]: [
      { role: 'admin' },
      { age: { [Op.gt]: 50 } }
    ]
  }
})
```

Métodos generados por las asociaciones

En Sequelize, cuando se crean asociaciones entre modelos (relaciones como uno a uno, uno a muchos, o muchos a muchos), Sequelize genera automáticamente métodos que facilitan la manipulación de estas asociaciones. Estos métodos permiten, por ejemplo, establecer, obtener, añadir, eliminar o contar las asociaciones entre las instancias de los modelos relacionados.

Aquí tienes una explicación de los métodos generados para cada tipo de asociación:

1. Relación Uno a Uno (hasOne y belongsTo):

- **hasOne**

- `get<Model>`: Obtiene la instancia relacionada.
- `set<Model>`: Asocia una instancia con otra.
- `create<Model>`: Crea y asocia una nueva instancia relacionada.

- **belongsTo**

- `get<Model>`: Obtiene la instancia a la que pertenece el modelo actual.
- `set<Model>`: Establece la asociación con una instancia existente.
- `create<Model>`: Crea y asocia una nueva instancia relacionada.

2. Relación Uno a Muchos (hasMany y belongsTo):

- **hasMany**
 - `get<Model>s`: Obtiene todas las instancias relacionadas.
 - `set<Model>s`: Establece múltiples instancias relacionadas (sobrescribe las existentes).
 - `add<Model>`: Añade una instancia relacionada.
 - `add<Model>s`: Añade múltiples instancias relacionadas.
 - `remove<Model>`: Elimina una instancia relacionada.
 - `remove<Model>s`: Elimina múltiples instancias relacionadas.
 - `count<Model>s`: Cuenta las instancias relacionadas.
 - `create<Model>`: Crea y asocia una nueva instancia relacionada.
- **belongsTo** (similar a la relación uno a uno)

3. Relación Muchos a Muchos (belongsToMany):

- **belongsToMany**
 - `get<Model>s`: Obtiene todas las instancias relacionadas.
 - `set<Model>s`: Establece múltiples instancias relacionadas (sobrescribe las existentes).
 - `add<Model>`: Añade una instancia relacionada.
 - `add<Model>s`: Añade múltiples instancias relacionadas.
 - `remove<Model>`: Elimina una instancia relacionada.
 - `remove<Model>s`: Elimina múltiples instancias relacionadas.
 - `count<Model>s`: Cuenta las instancias relacionadas.
 - `create<Model>`: Crea y asocia una nueva instancia relacionada.

Ejemplo usando los métodos generados por Sequelize:

Relación Uno a Muchos (Usuario - Proyecto):

Crear un usuario y asociarle proyectos.

```
async function crearUsuarioConProyectos() {
  const usuario = await Usuario.create({ nombre: 'Juan' })
  const proyecto1 = await Proyecto.create({ nombre: 'Proyecto 1' })
  const proyecto2 = await Proyecto.create({ nombre: 'Proyecto 2' })

  await usuario.addProyecto(proyecto1)
  await usuario.addProyecto(proyecto2)

  // También podrías usar createProyecto para crear y asociar en un
  solo paso
  const proyecto3 = await usuario.createProyecto({ nombre: 'Proyecto
3' })
}
```

```
}
```

Obtener todos los proyectos de un usuario:

```
async function obtenerProyectosDeUsuario() {
  const usuario = await Usuario.findOne({ where: { nombre: 'Juan' } })
})

const proyectos = await usuario.getProyectos()
console.log(proyectos.map(p => p.nombre))
}
```

Relación Muchos a Muchos (Usuario - Tarea):

Crear usuarios y tareas, y asociarlos:

```
async function crearTareaYAsociarUsuarios() {
  const tarea = await Tarea.create({ descripcion: 'Terminar documentación' })

  const usuario1 = await Usuario.create({ nombre: 'Ana' })
  const usuario2 = await Usuario.create({ nombre: 'Carlos' })

  // Añadir usuarios a la tarea
  await tarea.addUsuario(usuario1)
  await tarea.addUsuario(usuario2)

  // Añadir la misma tarea a otro usuario directamente
  const otroUsuario = await Usuario.create({ nombre: 'Maria' })
  await otroUsuario.addTarea(tarea)
}
```

Obtener todas las tareas asociadas a un usuario:

```
async function obtenerTareasDeUsuario() {
  const usuario = await Usuario.findOne({ where: { nombre: 'Ana' } })
  const tareas = await usuario.getTareas()
  console.log(tareas.map(t => t.descripcion))
}
```

Contar cuántos usuarios están asociados con una tarea:

```
async function contarUsuariosEnTarea() {
  const tarea = await Tarea.findOne({ where: { descripcion: 'Terminar documentación' } })
  const cantidadUsuarios = await tarea.countUsuarios()
}
```

```
console.log(`Usuarios asociados a la tarea: ${cantidadUsuarios}`)  
}
```